

Lista: Funções de Alta Ordem.

Nome: _____ Matrícula: _____

Exercícios baseados no capítulo 9 do livro Mordern Programming Languages.

Os exercícios abaixo devem possuir soluções a serem implementadas com uma única linha utilizando as funções `map`, `foldr`, e `foldl`. Você pode utilizar outras funções pré-definidas na linguagem, mas *não escreva nenhuma função adicional e não utilize recursão explícita*. Caso precise de alguma função auxiliar, utilize funções anônimas.

Por exemplo, se o problema dizer “escreva uma função `add2` que receba `int list` e retorne a mesma lista com o valor 2 adicionado a cada elemento”, a resposta deveria ser:

```
fun add2 x = map (fn a => a + 2) x;
```

Exercício 01. Escreva uma função `i12rl` do tipo `int list -> real list` que receba uma lista de inteiros e retorne uma lista dos mesmos números convertidos para o tipo real. Por exemplo, se você avaliar `i12rl [1, 2, 3]` você deve obter `[1.0, 2.0, 3.0]`.

Exercício 02. Escreva uma função `ordlist` do tipo `char list -> int list` que receba uma lista de caracteres e retorne a lista dos códigos inteiros desses caracteres. Por exemplo, se você avaliar `ordlist ['A', 'B', 'C']` você deve obter `[65, 66, 67]`.

Exercício 03. Escreva uma função `squarelist` do tipo `int list -> int list` que receba uma lista de inteiros e retorne a lista dos quadrados desses inteiros. Por exemplo, se você avaliar `squarelist [1, 2, 3, 4]` você deve obter `[1, 4, 9, 16]`.

Exercício 04. Escreva uma função `multpairs` do tipo `(int * int) list -> int list` que receba uma lista de pares de inteiros e retorne uma lista com

os produtos de cada par. Por exemplo, se a entrada for [(1, 2), (3, 4)], sua função deve retornar [2, 12].

Exercício 05. Escreva uma função `inclist` do tipo `int list -> int -> int list` que receba uma lista de inteiros e um incremento inteiro, e retorne a mesma lista de inteiros, mas com o incremento adicionado a cada um. Por exemplo, se você avaliar `inclist [1, 2, 3, 4] 10` você deve obter [11, 12, 13, 14]. Note que a função é curryficada.

Exercício 06. Escreva uma função `sqsum` do tipo `int list -> int` que receba uma lista de inteiros e retorne a soma dos quadrados desses inteiros. Por exemplo, se você avaliar `sqsum [1, 2, 3, 4]` você deve obter 30.

Exercício 07. Escreva uma função `bor` do tipo `bool list -> bool` que receba uma lista de valores booleanos e retorne o OU lógico de todos eles. Se a lista estiver vazia, sua função deve retornar `false`.

Exercício 08. Escreva uma função `band` do tipo `bool list -> bool` que receba uma lista de valores booleanos e retorne o E lógico de todos eles. Se a lista estiver vazia, sua função deve retornar `true`.

Exercício 09. Escreva uma função `bxor` do tipo `bool list -> bool` que receba uma lista de valores booleanos e retorne o OU exclusivo (XOR) de todos eles. (Deve retornar `true` se o número de valores `true` na lista for ímpar e `false` se for par). Se a lista estiver vazia, sua função deve retornar `false`.

Exercício 10. Escreva uma função `dupList` do tipo `'a list -> 'a list` cuja lista de saída é a mesma que a lista de entrada, mas com cada elemento da lista de entrada repetido duas vezes consecutivas. Por exemplo, se a lista de entrada for [1, 3, 2], a lista de saída deve ser [1, 1, 3, 3, 2, 2].

Exercício 11. Escreva uma função `mylength` do tipo `'a list -> int` que retorne o comprimento de uma lista. (Claro, você não pode usar a função `length` predefinida para isso).

Exercício 12. Escreva uma função `absreal` do tipo `int list -> real list` que receba uma lista de inteiros e retorne uma lista contendo os valores absolutos desses inteiros, convertidos para números reais.

Exercício 13. Escreva uma função `truecount` do tipo `bool list -> int` que receba uma lista de valores booleanos e retorne o número de `true` na lista.

Exercício 14. Escreva uma função `maxpairs` do tipo `(int * int) list -> int list` que receba uma lista de pares de inteiros e retorne a lista dos maiores elementos de cada par. Por exemplo, se você avaliar `maxpairs [(1, 3), (4, 2), (-3, -4)]` você deve obter `[3, 4, -3]`.

Exercício 15. Escreva uma função `myimplode` que funcione exatamente como a função predefinida `implode`. Em outras palavras, deve ser uma função do tipo `char list -> string` que receba uma lista de caracteres e retorne a `string` contendo esses mesmos caracteres na mesma ordem.

Exercício 16. Escreva uma função `concat` do tipo `'a list list -> 'a list` que receba uma lista de listas como entrada e retorne a lista formada pela concatenação das listas de entrada em ordem. Por exemplo, se a entrada for `[[1, 2], [3, 4, 5, 6], [7]]`, sua função deve retornar `[1, 2, 3, 4, 5, 6, 7]`. (Há uma função predefinida semelhante chamada `concat`, que, claro, você não deve usar).

Exercício 17. Escreva uma função `max` do tipo `int list -> int` que retorne o maior elemento de uma lista de inteiros. Sua função não precisa tratar o caso da lista estiver vazia.

Exercício 18. Escreva uma função `min` do tipo `int list -> int` que retorne o menor elemento de uma lista de inteiros. Sua função não precisa tratar o caso da lista estiver vazia.

Exercício 19. Escreva uma função `member` do tipo `'a * 'a list -> bool` para que `member (e, L)` seja `true` se e somente se `e` for um elemento da lista `L`.

Exercício 20. Escreva uma função `append` do tipo `'a list -> 'a list -> 'a list` que receba duas listas e retorne o resultado de anexar a segunda no final da primeira. Por exemplo, `append [1, 2, 3] [4, 5, 6]` deve avaliar para `[1, 2, 3, 4, 5, 6]`. Não use utilitários predefinidos de concatenação, como o operador `@` ou a função `concat`. Note que a função é curryficada.

Exercício 21. Defina uma função `less` do tipo `int * int list -> int list` para que (e, L) seja uma lista de todos os inteiros em L que são menores que e (em qualquer ordem).

Exercício 22. Escreva uma função `evens` do tipo `int list -> int list` que receba uma lista de inteiros e retorne a lista de todos os elementos pares da lista original (na ordem original). Por exemplo, se você avaliar `evens [1, 2, 3, 4]`, você deve obter `[2, 4]`.

Exercício 23. Escreva uma função `convert` do tipo `('a * 'b) list -> 'a list * 'b list` que converta uma lista de pares em um par de listas, preservando a ordem dos elementos. Por exemplo, `convert [(1, 2), (3, 4), (5, 6)]` deve avaliar para `([1, 3, 5], [2, 4, 6])`.

Exercício 24. Defina uma função `mymap` com o mesmo tipo e comportamento de `map`, mas sem usar `map`. (Note que ainda deve ser uma função de uma linha: use `foldl` ou `foldr`).

Exercício 25. Represente um polinômio usando uma lista de seus coeficientes (reais), começando com o coeficiente constante e indo apenas até onde for necessário. Por exemplo, $3x^2 + 5x + 1$ seria representado como a lista `[1.0, 5.0, 3.0]` e $x^3 - 2x$ como `[0.0, -2.0, 0.0, 1.0]`. Escreva uma função `eval` do tipo `real list -> real -> real` que receba um polinômio representado desta forma e um valor para x e retorne o valor desse polinômio. Por exemplo `eval [1.0, 5.0, 3.0] 2.0` deve avaliar para 23.0, pois quando $x = 2$, $3x^2 + 5x + 1 = 23$.